Furnace App - Administrative Dashboard

Introduction **2**
- Overview 3
  - Features 3
  - Schema 3
  - Packages 3
- Getting Started 3

Admin Dashboard Home **5**

View Users **6**
- View Students 6
  - Students List 6
  - Students Detail 6
- View Administrators 6
  - Administrator List 6
  - Administrator Detail 6

View Rooms **6**
- Rooms List 6
- Room Details 6
- Room - Add 6
- Room - Edit 6

Analytics **6**
- Rooms List 6
- Room Details 6

Admin Settings **6**
- Admin Settings 6
- Admin Settings Edit 6

Introduction

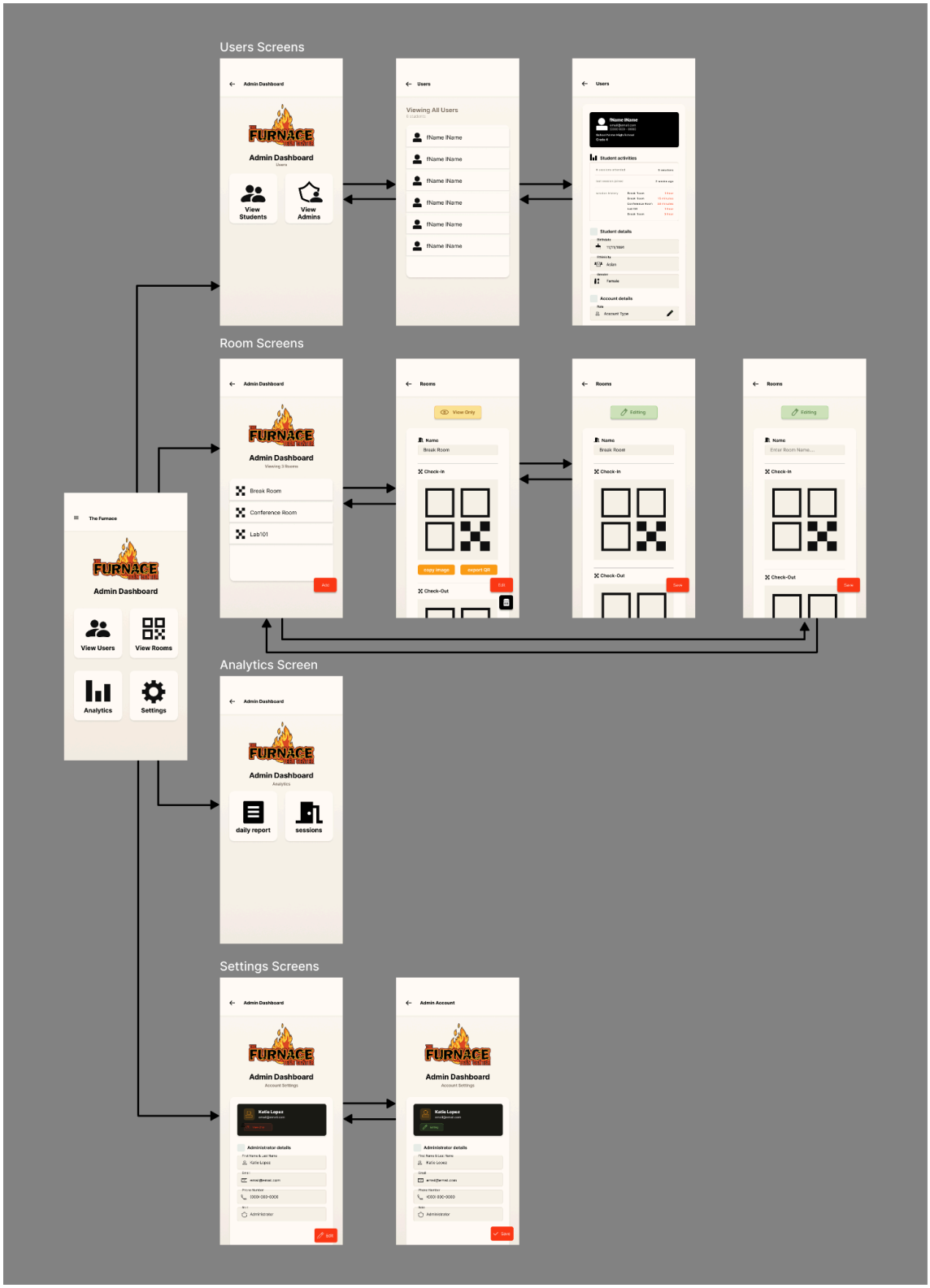


Figure 1: Above is the general flow of the admin dashboard.

See Furnace App Read Me for student flow. This document will focus on the flow and features of the administrator.

Overview

The administrator dashboard allows admins and staff to manage student accounts, rooms, view analytics, and manage personal account settings. Only the site supervisor can be an admin. An admin can add staff who also have admin privileges.

Not all features and screens have been implemented. All features and screens that have not been implemented are labeled with (To be implemented).

Features

- View all students and administrators enlisted in the app
- Room management
- Analytics for exporting data (not implemented)
- Admin profile management

Packages

- QR Code generation | qr_flutter
- Exporting QR Code into a pdf | pdf

Getting Started

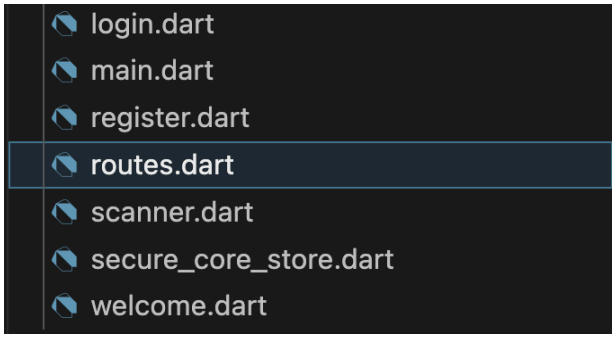
Logging in as Administrator

Currently, there are two ways of logging in as administrator depending on if you are using mobile or web.

Mobile

Only two things need to change in routes.dart.

1. Navigate to routes.dart



2. On line 36, change home route from “/” to “/admin-home”

```
initialLocation: '/admin-home',
```

3. Comment out line 40 to line 54

```
        // final isAuthPage =
        //     location == '/' || location == '/login' || location ==
  '/register';

        // if (!isLoggedIn && !isAuthPage) {
        //   return '/login';
        // }

        // if (isLoggedIn && isAuthPage) {
        //   return isAdmin ? '/admin-home' : '/welcome';
        // }

        // // Prevent non-admin users from manually navigating to admin routes
        // if (location.startsWith('/admin') && !isAdmin) {
        //   return '/welcome';
        // }
```

routes.dart should look like this at the end.

```
  return GoRouter(
    initialLocation: '/admin-home',
    redirect: (context, state) {
      final location = state.uri.toString();

      // final isAuthPage =
```

```
    //     location == '/' || location == '/login' || location == '/register';

    // if (!isLoggedIn && !isAuthPage) {
    //   return '/login';
    // }

    // if (isLoggedIn && isAuthPage) {
    //   return isAdmin ? '/admin-home' : '/welcome';
    // }

    // // Prevent non-admin users from manually navigating to admin routes
    // if (location.startsWith('/admin') && !isAdmin) {
    //   return '/welcome';
    // }

    return null;
  },
```

OR

1. In your .env, add a new variable, ADMIN_TEST_EMAIL and set it to an email you logged in with.

```
PARSE_APP_ID=nxvY8VUpLTm
PARSE_CLIENT_KEY=AEyU2ez
PARSE_SERVER_URL=https://parseapi.back4app.com
ADMIN_TEST_EMAIL=someemail@gmail.com
```

2. Log in using this email, then re-run the app. You should be brought to the admin home page (actual implementation for logging in as an admin will need to be properly implemented)

Admin Dashboard Home

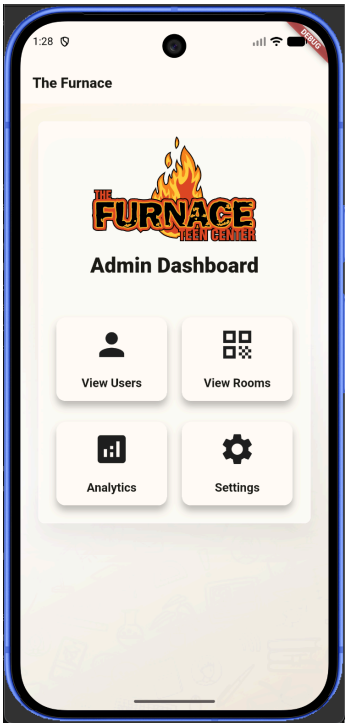


Figure 2: Admin Dashboard. Admin dashboard is the home screen for the admin interface.

View Users

Allows users to view a list of students or admins.

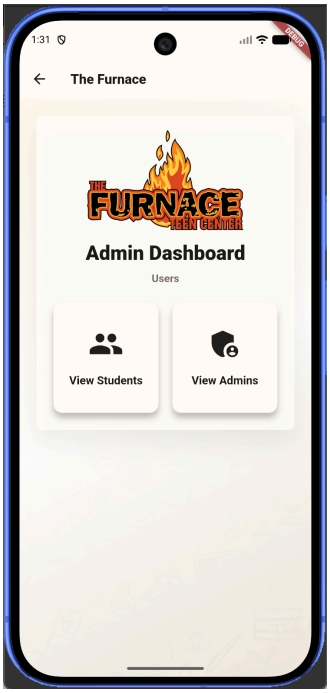


Figure 3. Users Screen. Users screen allows administrators to either view a list of users or a list of administrators.

View Students

Students List

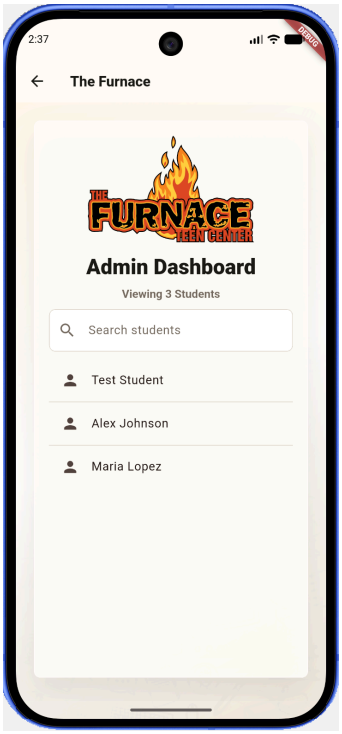


Figure 4. Students List screen. Displays a list of students enrolled in the app.

Features

- Filter students by name via search bar
- Display total number of students in database

Students Detail

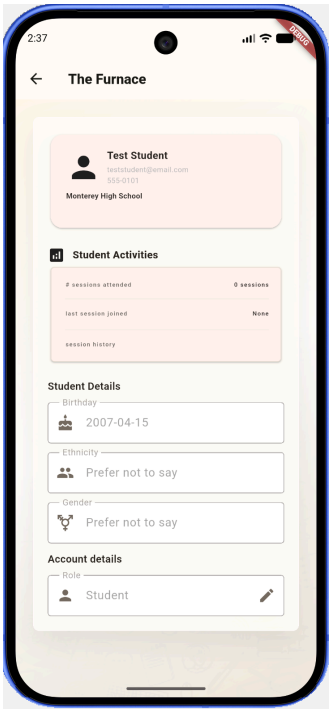


Figure 5. Students Detail screen. Displays details for a student that has been chosen from the student list.

Features

- Displays a student's name, email, school, phone number, birthdate, ethnicity, gender, and role. (Role is currently hard-coded to display "Student")
- Displays recent student activity. Details a student's total number of sessions, the last time they checked into a room, and the 5 most recent rooms they checked into.
- Edit student role (To be implemented)

View Administrators (To be implemented)

Administrator List (To be implemented)

Administrator Detail (To be implemented)

View Rooms

Rooms List

The Room List screen allows administrators to view a list of rooms they have created.

Features

- View names of each room that has been created
- Navigate to Room Details
- Create a new room

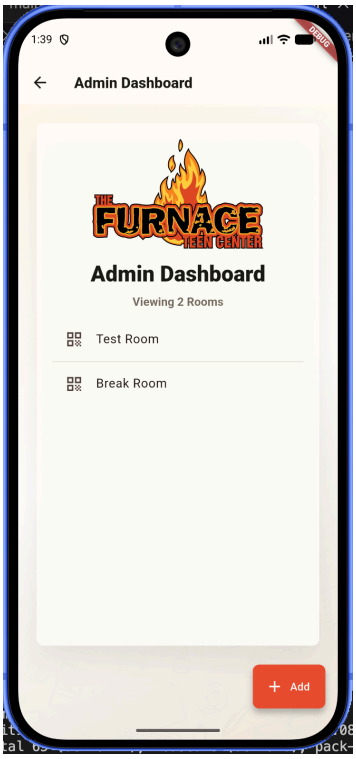


Figure 6. Room List Screen displaying all rooms that the administrator has created.

Room Details

The Room Details screen displays the room name and two QR codes, one for check in and one for check out. The QR generation uses the qr_flutter library. The QR code contains the room name and either qrRoomValue (for check in) or qrCheckoutValue (for check out).

Features

- View room details, including room name and generated QR codes
- Export QR code to PDF
- Copy QR code image (To be implemented)
- Edit room details (To be implemented)

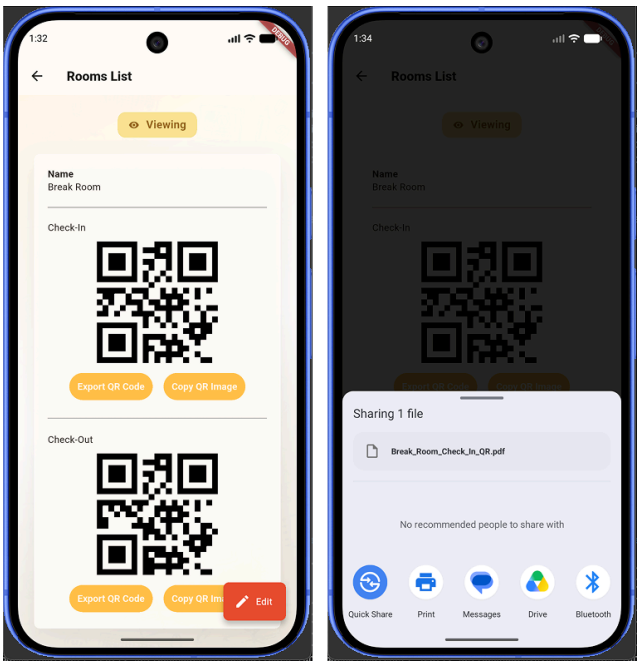


Figure 7. Left: View Room Screen displaying room name, check in QR code and checkout QR code. Right: Export to PDF screen after pressing the Export button

Room - Add (To be implemented)

The Add Room screen allows administrators to create a new room. The QR code is autogenerated once the room name is entered or the save button is pressed. The QR generation uses the qr_flutter library.

There should be validation checking that the room name is not empty. There should also be a way to cancel if a room no longer wants to be added.

This screen will have a similar design to the View Room Screen.

Features

- Add a new room
- Validate that room name
- Autogenerate check in and check out QR code after inputting room name

User Workflow

1. Press on “Add” button from room list
2. Input a room name
3. QR code is either autogenerated here or after pressing save
4. Press Save and view new room in room list

Room - Edit (To be implemented)

The Edit Room Screen allows administrators to edit a room. An administrator must select a room they wish to edit. From there, they can click on the edit button to edit the room. The only thing that the administrator can edit is the room name. The QR code is automatically generated with the qr_flutter library.

There should be validation checking that the room name is not empty. There should also be a way to cancel if a room no longer wants to be added.

This screen will have a similar design to the View Room Screen.

Features

- Edit an existing room name
- Validate room name
- View QR codes that are autogenerated

User Workflow

1. Select a room you would like to edit
2. Press on “Edit” button

3. Change room name or delete room
4. Press Save and view edited room in room list

Analytics (To be implemented)

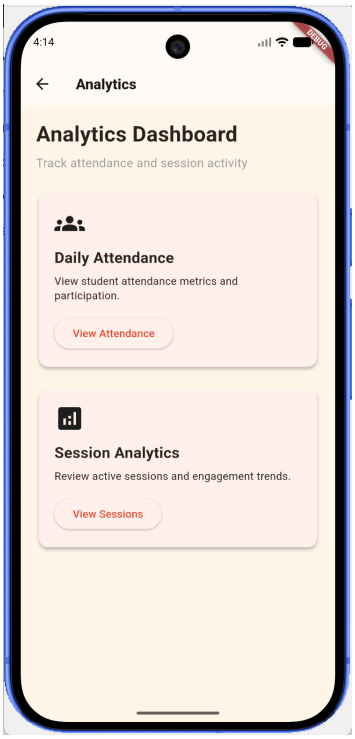


Figure 8: Admin Analytics screen displaying Daily Attendance and Sessions Analytics buttons

View Attendance (To be implemented)

The Attendance Screen allows administrators to view a list of students who have attended a session, based on the date. This may change as change. Below is a preliminary design of the Attendance Screen and flow.

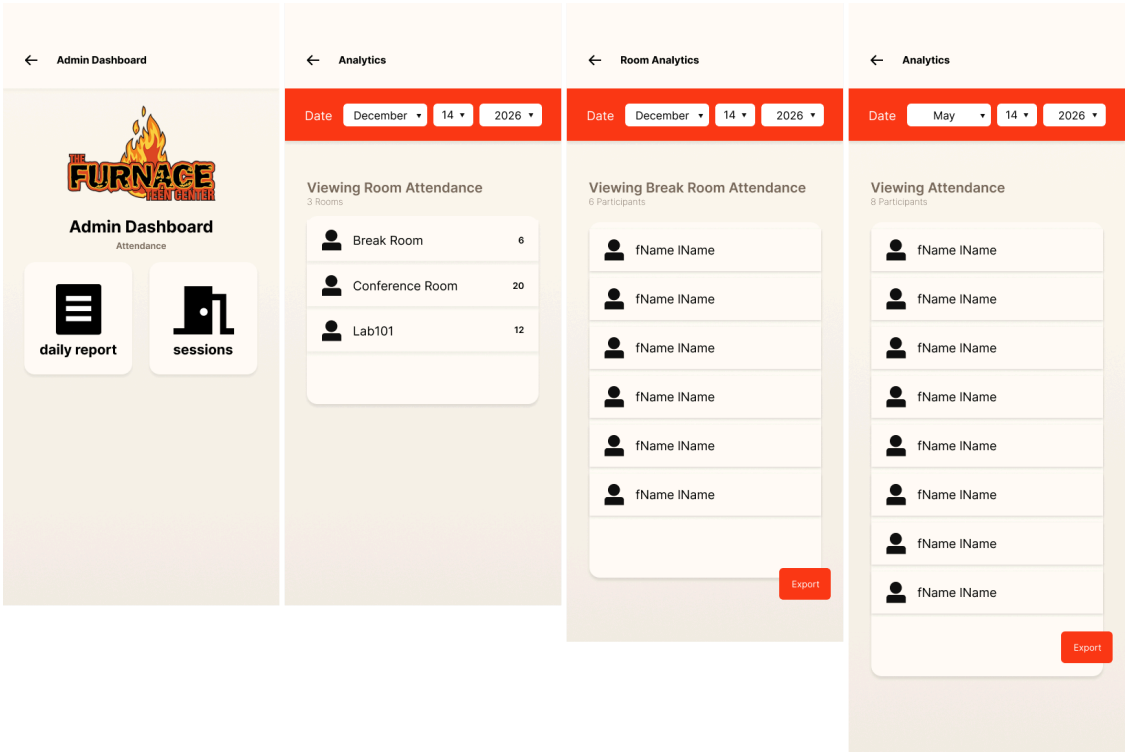


Figure 9: Prototype design for Attendance and Sessions Screen.

Session Analytics (To be implemented)

See Figure 9.

Admin Settings

Admin Settings

The Admin Settings page allows administrators to view their account information within the application. Information displayed includes the administrator's name, email address, phone number, and role. This screen serves as the primary location for reviewing account information and provides access to editing functionality.

Features:

- View administrator profile information
- View email address
- View phone number
- View administrator role
- Navigate to Edit Settings screen

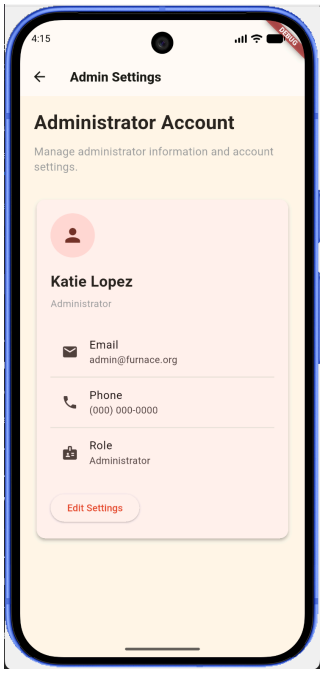


Figure 10: Admin Settings screen displaying administrator account information.

Admin Settings Edit

The Admin Settings Edit page allows administrators to modify account information. Form validation is implemented to ensure data is entered correctly before changes are saved.

Features:

- Edit administrator name
- Edit administrator email address
- Edit administrator phone number
- Edit administrator role
- Save updated account information
- Reset fields to their original values
- Cancel editing and return to the Admin Settings screen
- Form validation for required fields
- Email format validation
- Phone number validation

Validation Rules:

- All fields are required before saving.
- Email addresses must contain a valid email format.
- Phone numbers must contain at least 10 digits.
- Validation messages are displayed when invalid information is entered.

User Workflow:

1. Open Admin Settings.
2. Select Edit Settings.
3. Update account information.
4. Select Save to apply changes.
5. Select Reset to restore original values.
6. Select Cancel to return without saving changes.

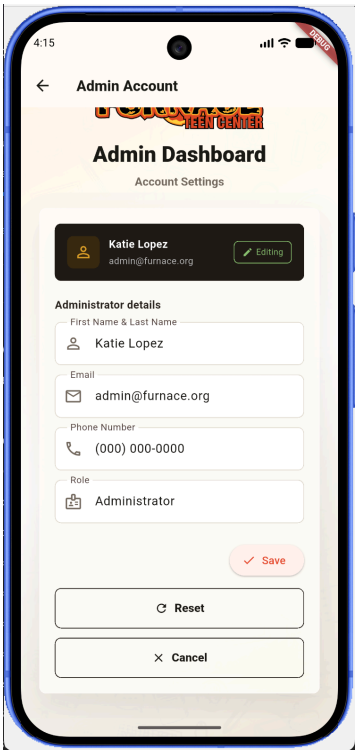


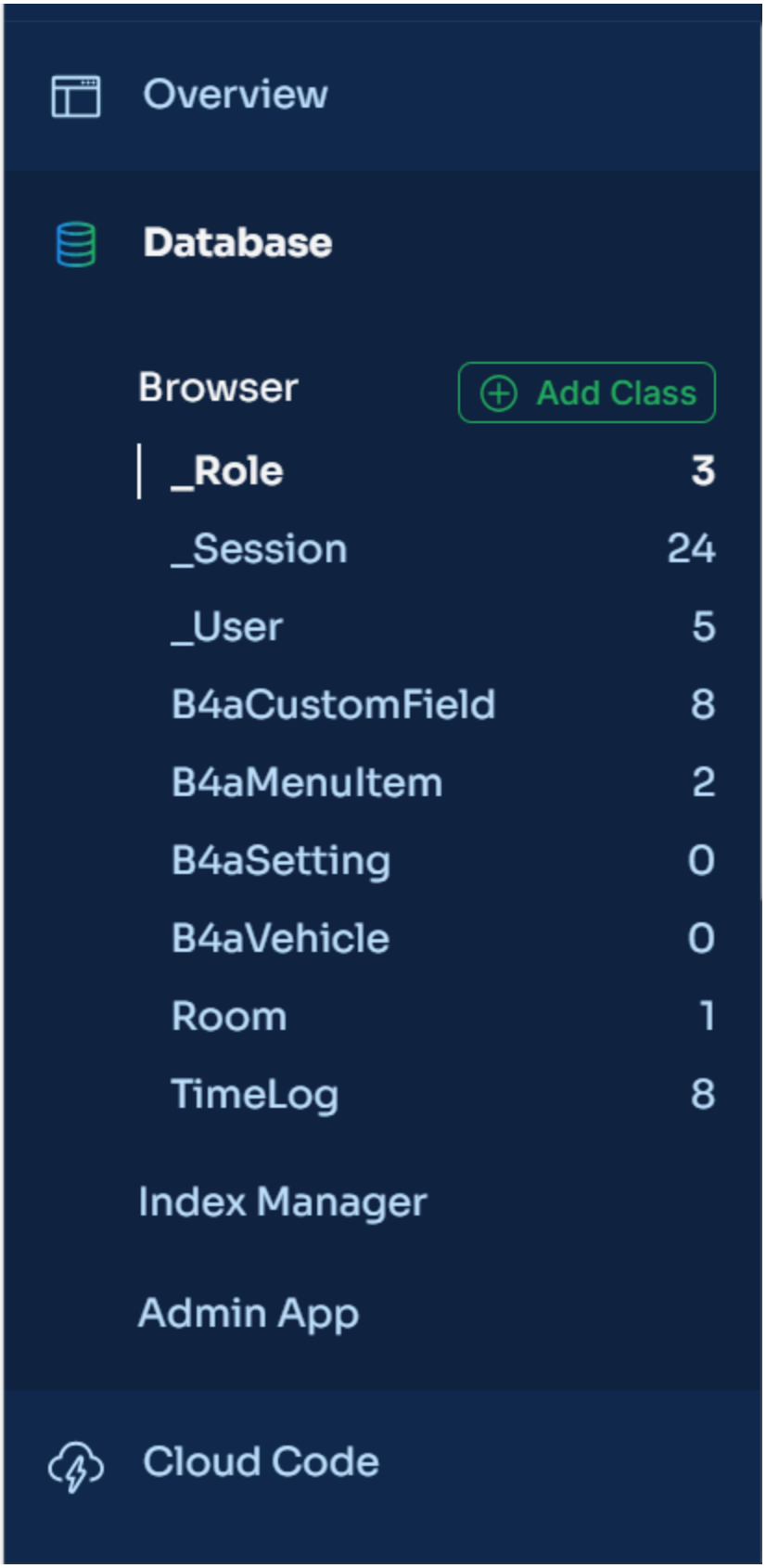
Figure 11: Admin Settings Edit screen displaying editable administrator account information.

GitHub Actions - CI

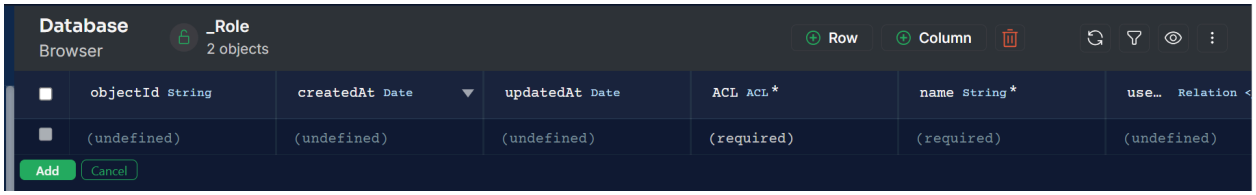
Files for CI will be located in .github/workflows/. When a pull request is opened or a commit is merged into main, GitHub will automatically run and build unit tests, informing you of any failing tests or build errors. GitHub will also run `flutter analyze` and leave comments in your PR detailing any code quality issues.

How To: Add an Admin role in Back4App

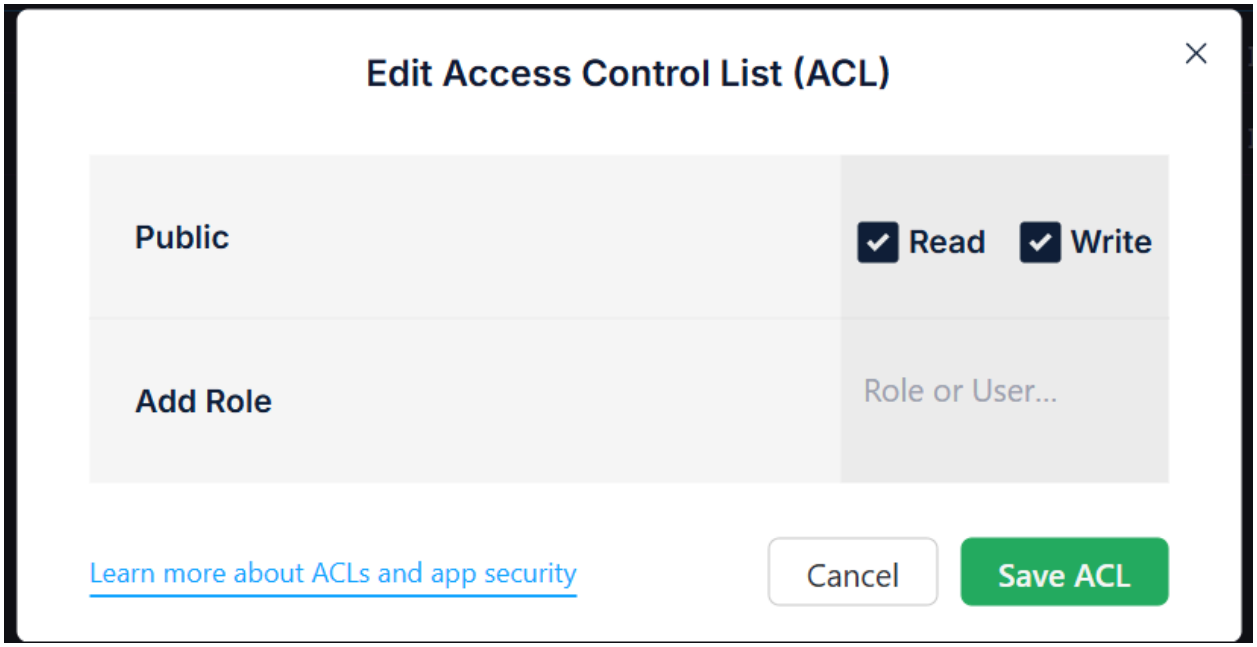
1. In Back4App, go to Database > browser > _Role


Overview
Database
Browser
Add Class
_Role 3
_Session 24
_User 5
B4aCustomField 8
B4aMenuItem 2
B4aSetting 0
B4aVehicle 0
Room 1
TimeLog 8
Index Manager
Admin App
Cloud Code

2. Create a new row for this object



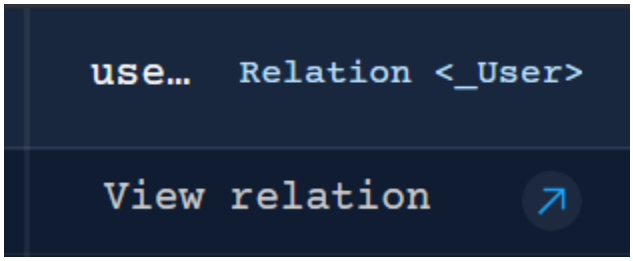
3. In the ACL field, give the appropriate read or write permissions.



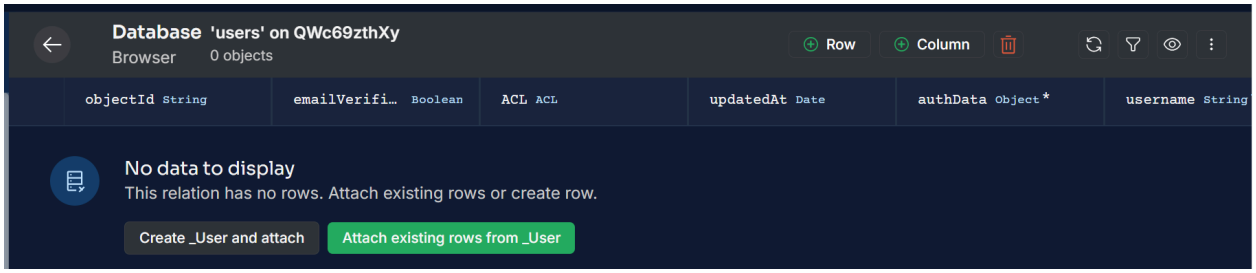
4. In the name field, add a name for the role (e.g. “Staff”)
5. After clicking “add”, it should create a new row in the _Role object



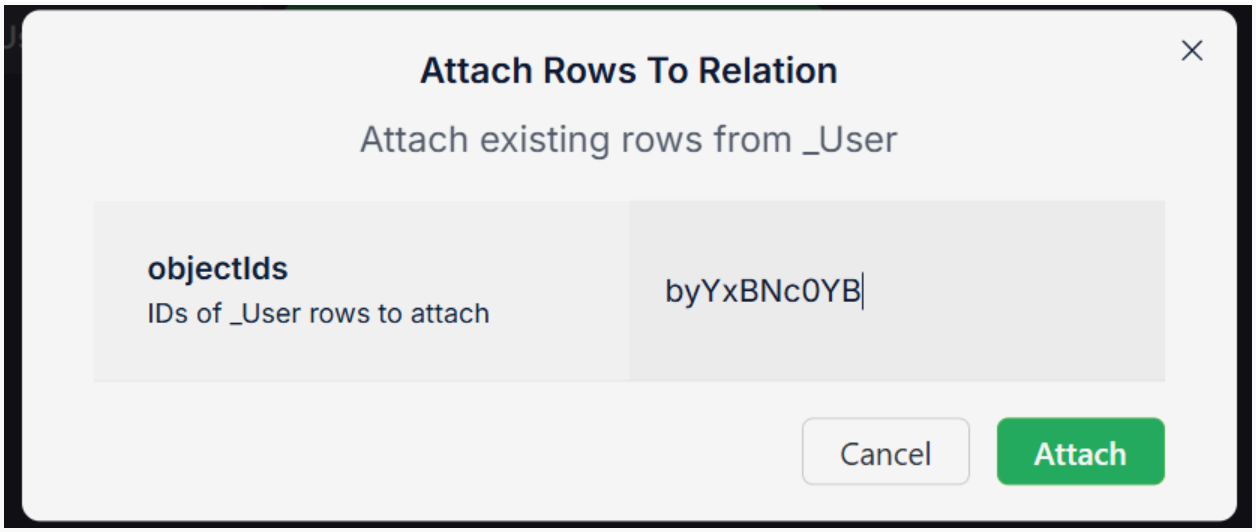
6. To add this role to a user, click on “View relation” for the Relation<User> field



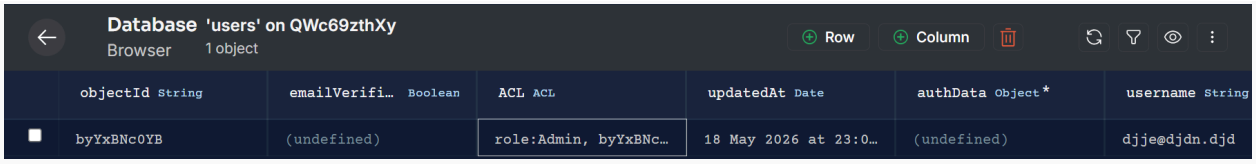
7. Click on “Attach Existing rows from _User”



8. Copy the objectId of the user you want to add to this new role



9. Click on “Attach” and the user should be given this new role



Official case study from Flutter on MVVM architecture

- [UI layer case study]